

مترجم الأردوينو العربي

ArArduino Compiler — Engineering & Academic Documentation

لغة برمجة عربية موجهة للأنظمة المدمجة ومعالجات AVR

مبني على Python 3 · LLVM IR · ANTLR4

المشرفون:

د. عبد العظيم عموري
م. عماد قرحيلي
م. علي سليمان

المقدمون:

عبد الهادي نشواتي
محمد ثائر عرابي
سعيد بقونس

فهرس المحتويات

- المقدمة والتعريف بالمشروع
- بنية ومعمارية المشروع العامة
- الواجهة الأمامية للمترجم
- التحليل الدلالي ونظام الأنواع
- الواجهة الوسطى وتحسين الكود الوسيط
- الواجهة الخلفية وتوليد كود الآلة والربط
- طبقة بيئة التشغيل ومغلفات لغة ++C
- بيئة التطوير المتكاملة العربية
- واجهة السائق، الاختبارات والرسم البصري
- الخلاصة والآفاق المستقبلية
- مستودعات المشروع

1. المقدمة والتعريف بالمشروع

1.1. فكرة المشروع وأهدافه الأكاديمية والعملية

تتمحور فكرة هذا المشروع حول تصميم وبناء مترجم متكامل (End-to-End Compiler) للغة برمجة مخصصة مبتكرة ذات بناء نحوي مستوحى من اللغة العربية، وموجهة للتحكم الدقيق بالمعالجات الدقيقة والأنظمة المدمجة (Embedded Systems) وتحديداً لوحات الأردوينو (Arduino Uno / ATmega328P).

لا يقتصر المترجم على ترجمة النصوص، بل يمتد ليكون نظام تعريف متقاطع (Cross-Compiler) حقيقي؛ حيث يستقبل الكود المصدري العربي النظيف، ويمرره عبر سلسلة من المعالجات الرياضية والدلالية، وصولاً إلى توليد الكود لـ LLVM IR وتجميعه إلى لغة الآلة المباشرة لمعالجات AVR دون الحاجة لوسيط مثل لغات ++C/C.

الأهداف الأكاديمية:

- فهم بنية المترجمات الحديثة:** تطبيق المعايير الأكاديمية المتبعة في بناء المترجمات بدءاً من مرحلة معالجة النصوص الطبيعية (NFC Normalization) وحتى التجميع العتادي المباشر.
- إدارة وتحليل الأنواع دلاليًا (Type & Scope Semantics):** تصميم هياكل بيانات متقدمة لإدارة مجالات الرؤية (Scopes) وفحص توافق الأنواع (Type Checking) دلاليًا لمنع وقوع كوارث منطقية في الأنظمة منخفضة المستوى.
- تحسين الكود الوسيط للعتاد المحدود:** دراسة وتطبيق تمريرات تحسين الأداء والحجم (Size & Speed Optimization) لمعالجات 8-بت ذات الذاكرة المحدودة جداً.

الأهداف العملية:

- توطين التقنية وتسهيل التعليم:** إزالة حاجز اللغة الإنجليزية أمام الطلاب والمبتدئين في العالم العربي لتمكينهم من تعلم البرمجة عتادياً والتعامل مع الإلكترونيات والمتحكمات الدقيقة بلغة قريبة من لغتهم الأم.
- الأداء العتادي العالي (Bare-Metal Efficiency):** توفير لغة تترجم مباشرة إلى لغة الآلة بكفاءة تضاهي سرعة وحجم الكود المكتوب بلغة C، مع إتاحة التحكم المباشر بمسجلات العتاد (Hardware Registers).

1.2. القيمة العلمية للمترجمات الموجهة باللغات المحلية

تمثل المترجمات المكتوبة بلغات غير اللاتينية تحدياً علمياً وبحثياً فريداً في علوم الحاسوب. وتكمن القيمة العلمية لهذا المشروع في النقاط التالية:

- استقلالية المعالجة النحوية عن اللغة:** إثبات أن النظريات النحوية الرياضية (مثل القواعد خالية السياق CFG وعمليات الإعراب LL1) هي نماذج رياضية مجردة صالحة للتطبيق على أي لغة بشرية بغض النظر عن اتجاه الكتابة أو رسم الحروف.
- معالجة مشاكل اليونيكود (Unicode Challenges in Compilers):** يواجه المترجم العربي تحديات تقنية فريدة تتعلق بترميز الحروف (UTF-8)، وعلامات التشكيل، وكشيدات التطويل، والهمزات، والحروف المركبة (Ligatures)، بناء أنبوب تسوية نصوص (Normalization Pipeline) قبل التحليل المعجمي يعد مساهمة علمية في معالجة اللغات الطبيعية (NLP) المدمجة بالمترجمات.

- **إمكانية التوسع البرمجي (Extensibility):** تصميم واجهة برمجية موحدة للزوار (ASTVisitor) تتيح إضافة مميزات برمجية جديدة للغة بشكل مستقل تماماً دون الحاجة لتغيير هيكلية الشجرة (تطبيقاً لمبدأ Open-Closed المعماري).

1.3. متطلبات النظام والبيئة التشغيلية الافتراضية

المتطلبات البرمجية الأساسية:

- **بيئة تشغيل لغة بايثون (+Python 3.10):** البيئة الأساسية التي تحتضن كود الشجرة والتحليل والتحسين وتوليد الكود.
- **بيئة تشغيل جافا (+JRE 11):** مطلوبة حصرياً لتشغيل أداة ANTLR4 لتوليد ملفات بايثون المعجمية والنحوية من ملفات القواعد .g4 .
- **مكتبة ANTLR Python Runtime (الإصدار 4.13.2):** للربط بين كود بايثون والمفسر المولد.
- **مكتبة LLVMlite (الإصدار 0.47.0):** لربط المترجم بمكتبات LLVM الديناميكية لتوليد وتحسين كود الـ Intermediate Representation (IR).
- **برمجيات ومكتبة Graphviz (اختياري):** مطلوبة لتصدير ورسم شجرة القواعد المجردة رسوماً وتفاعلياً.

الجزء الثاني

بنية ومعمارية المشروع العامة (System Architecture)

2. بنية ومعمارية المشروع العامة

2.1. الهيكل التنظيمي للمجلدات والملفات (Directory Tree)

يتبع المشروع معمارية مجزأة وصارمة تفصل الواجهة الأمامية تماماً عن المنظومة الدلالية والمرحلة الخلفية:

```

ArArduinoCompiler/
├── main.py                # المحرك الرئيسي المنسق لخطوات التصريف (Compiler Driver)
│                           (test_arduino.txt مُعرّف حالياً بشكل ثابت على)
├── build.py               # سائق العمليات الشامل للبناء والرفع
├── normlize.py            # وحدة تصفية وتسوية النصوص العربية (Text Normalization)
├── test_arduino.txt       # كود الأردوينو العربي المراد تصريفه واختباره
├── requirements.txt       # وثيقة المكتبات وبيانات الاعتماديات البرمجية
├── debug.ll               # الخام المولد قبل التحسين LLVM IR ملف كود
├──
├── ast_dir/               # حزمة شجرة القواعد المجردة (AST Package)
│   ├── __init__.py
│   ├── nodes.py           # AST فئات كتل وعقد الـ
│   └── visitor_interface.py # AST الواجهة البرمجية المجردة لجميع زوار الـ
├──
├── frontend/              # حزمة الواجهة الأمامية (ANTLR Grammars & Parsers)
│   ├── __init__.py
│   ├── ArArduinoLexer.g4   # ملف قواعد الماسح المعجمي (Lexer Rules)
│   ├── ArArduinoParser.g4  # ملف قواعد المحلل النحوي (Parser Rules)
│   ├── ArArduinoLexer.py   # الماسح المعجمي المولد بلغة بايثون
│   ├── ArArduinoParser.py  # المحلل النحوي المولد بلغة بايثون
│   ├── ArArduinoParserVisitor.py
│   └── ast_builder.py      # زائر البناء المخصص (AST Builder)
├──
├── semantic/              # حزمة التحليل الدلالي وفحص الأنواع
│   ├── __init__.py
│   ├── symbols.py          # فئات سجلات الرموز
│   ├── environment.py      # نظام إدارة البيانات والنطاقات المعجمية
│   ├── types_system.py     # Singletons فئات تمثيل الأنواع البرمجية كـ
│   ├── builtins_registry.py # ATmega328P سجل دوال وثوابت ومسجلات العتاد
│   └── semantic_analyzer.py # زائر التحليل الدلالي وفحص توافق الأنواع
├──
├── backend/               # حزمة المرحلة الخلفية والتحسين
│   ├── __init__.py
│   ├── ir_generator.py     # AVR الموجه لمعالجات LLVM IR مولد كود
│   ├── avr_target.py       # AVR (ATmega328P) إعداد آلة الهدف لـ
│   ├── codegen.py          # AVR إلى كود تجميع/كائن LLVM IR تحويل
│   ├── linker.py           # hex الربط وتوليد ملف
│   └── optimizer.py        # ومولد الكائنات الثنائية LLVM IR محسن
├──
├── runtime/
│   └── arduino_glue.cpp     # معالج الذعر، main() دالة، C مغلفات
├──
├── samples/
│   ├── blink.txt
│   ├── serial_hello.txt
│   └── panic_test.txt
├──
├── tools/                  # الأدوات والملفات التنفيذية المساعدة
│   ├── antlr-4.13.2-complete.jar
│   ├── setup_avr_toolchain.py
│   ├── build_core.py
│   └── init_antlr_alias.bat

```

2.2. أدوات ومكتبات التطوير المستخدمة

- **أداة (ANTLR4 (Another Tool for Language Recognition):** تُستخدم لقراءة قواعد المترجم النحوية المكتوبة في ملفات g4. وتوليد الماسح المعجمي والمحلل النحوي تلقائياً بلغة بايثون، مما يوفر محركاً عالي الدقة لإعراب النصوص.

- **مكتبة LLVM Lite:** تمثل الجسر البرمجي للغة بايثون للتحكم بـ LLVM. نستخدمها لتوليد كود الـ LLVM IR، وتطبيق التحسينات، وتجميع الكود إلى كائنات ثنائية (.o).
- **أداة Graphviz:** أداة رسم بياني تتيح تمثيل كائنات الـ AST وتحويلها رسوماً إلى شجرات هندسية ملونة لتسهيل التحقق البصري من صحة بنية الشجرة.

2.3. دورة حياة المترجم وتدفق البيانات (Compiler Pipeline)

حذف التشكيل، ← [normalize.py] تسوية وتنظيف النص [▼ | [test_arduino.txt) الكود العربي المصدر] Tokens تفكيك النص إلى سلسلة من الرموز ← [Lexer.py) الماسح المعجمي [▼ | التطويل، توحيد الهمزات باني الشجرة [▼ | Parse Tree التحقق النحوي وإنشاء ← [Parser.py) المحلل النحوي [▼ | (semantic_analyzer) المحلل الدلالي [▼ | ++التقاط السطور، تفكيك س، AST توليد ← [ast_builder.py) LLVM IR إنتاج كود ← [ir_generator.py) مولد الكود [▼ | فحص النطاقات وتوافق الأنواع والتسميم ← [المخرجات [▼ | لمعالجات 8-بت -Os تطبيق تحسينات ← [AVR (optimizer.py) المحسن لـ [▼ | الخام [ar_arduino_ast.png / output.o) النهائية

الجزء الثالث

الواجهة الأمامية للمترجم (The Front-End)

3. الواجهة الأمامية للمترجم

تعتبر الواجهة الأمامية بمثابة بوابة العبور للمترجم؛ حيث تستقبل النص البرمجي الخام المكتوب باللغة العربية، وتقوم بفحصه وتفكيكه تمهيداً لبناء هيكل بيانات منظم في الذاكرة. تتكون هذه الواجهة من أربع مراحل متتالية: تسوية النصوص، التحليل المعجمي، التحليل النحوي، وبناء شجرة القواعد المجردة (AST).

3.1. مرحلة تسوية وتطهير النصوص العربية (Arabic Text Normalization Pipeline)

```
# normalize.py
import re
import unicodedata

def normalize_arabic_text(text: str) -> str:
    # 1. التسوية المعيارية (NFC)
    nfc_text = unicodedata.normalize('NFC', text)
    # 2. إزالة علامات التشكيل وكشيدة التطويل
    clean_text = re.sub(r'[\u064B-\u065F\u0640]', '', nfc_text)
    # 3. توحيد الهمزات
    clean_text = re.sub(r'[\i\I]', '', clean_text)
    return clean_text
```

3.1.1. التسوية المعيارية وتوحيد الحروف المركبة (NFC Composed)

3.3. المحلل النحوي (The Parser - ArArduinoParser.g4)

3.3.1. قواعد اللغة خالية السياق (CFG Rules Hierarchy)

```
program      : importStmt* declaration* EOF ;
importStmt   : IMPORT STRING SEMI ;
declaration: varDecl | funDecl ;
varDecl      : VAR ID COLON type ASSIGN expression SEMI ;
```

لحل مشكلة التراجع الأيسر وفرض أولويات العمليات وفق شروط **LL(1)**، تم بناء هرمية التعبيرات من الأضعف أولوية إلى الأقوى:

```
orExpr → andExpr → bwOrExpr → bwXorExpr → bwAndExpr → relExpr → shiftExpr → addExpr →
mulExpr → unaryExpr → primary
```

```
shiftExpr : addExpr ((SHL | SHR) addExpr)* ;
```

3.3.2. حل مشاكل التراجع الأيسر والغموض (LL1 Compliance)

```
funDecl : FUNCTION funcBody ;
funcBody
  : SETUP LPAREN RPAREN COLON type block
  | LOOP  LPAREN RPAREN COLON type block
  | ID    LPAREN params? RPAREN COLON type block
  ;
```

تم حل مشكلة الـ Dangling Else بفرض استخدام الأقواس المعقوفة {} إجبارياً على كافة فروع جمل الشرط والتكرار.

3.4. باني شجرة القواعد المجردة (AST Builder - ast_builder.py)

3.4.1. فئات كتل الـ AST وتصميمها الكائني (ast_dir/nodes.py)

```
# ast_dir/nodes.py
@dataclass
class ASTNode(ABC):
    line: int = 0
    column: int = 0
    resolved_type: Any = None # حقل تخزين نوع البيانات المستنتج دلاليّاً لاحقاً

    @abstractmethod
    def accept(self, visitor: 'ASTVisitor'):
        pass
```

يمثل حقل `resolved_type` الجسر الرابط بين التحليل الدلالي وتوليد الكود؛ حيث يتم حقن النوع المستنتج مباشرة في العقدة لتستخدمه الواجهة الخلفية دون إعادة تقييمه.

3.4.2. تفكيك التبسيط النحوي برمجياً (Syntactic Desugaring)

يقوم باني الشجرة بتطبيق **التفكيك النحوي (Desugaring)** صامتاً على العوامل المركبة:

• تحويل $s++ \leftarrow s = s + 1$

• تحويل $s += o \leftarrow s = s + o$

يخرج هذا التعديل كـ `AssignNode` نظيف يحتوي على `BinOpNode` ، مما يلغي الحاجة لمعالجة خاصة في المراحل اللاحقة.

3.4.3. التقاط وحقق البيانات الوصفية للعقد (Metadata: Line & Column)

```
def set_metadata(self, node: ASTNode, ctx):
    if node and ctx and ctx.start:
        node.line = ctx.start.line
        node.column = ctx.start.column
    return node
```

ملاحظة تقنية: تم حل مشكلة الخط في `@dataclass` عبر إجبار المترجم على استخدام الأسماء المفتاحية صراحةً (Keyword Arguments) أثناء إنشاء كتل العقد، مثل: `ProgramNode(declarations=...)`.

الجزء الرابع

التحليل الدلالي ونظام الأنواع (Semantic Analysis & Type Checking)

4. التحليل الدلالي ونظام الأنواع

تمثل هذه المرحلة "دماغ المترجم"؛ حيث يتم التحقق من أن الكود المكتوب متناسق منطقياً، وأن المتغيرات معرفة في نطاقاتها الصحيحة، وأن العمليات الحسابية والمنطقية تحترم قوانين الأنواع الخاصة باللغة.

4.1. سجل الكيانات المدمجة للأردوينو ومنافذ العتاد (Built-ins & Register Registry)

4.1.1. الدوال القياسية والثوابت المدمجة


```
# semantic/builtins_registry.py
CORE_CONSTANTS = {
    "عالي": ConstantSymbol("عالي", INT_TYPE), # HIGH
    "منخفض": ConstantSymbol("منخفض", INT_TYPE), # LOW
    "مدخل": ConstantSymbol("مدخل", INT_TYPE), # INPUT
    "مخرج": ConstantSymbol("مخرج", INT_TYPE), # OUTPUT
}
CORE_FUNCTIONS = {
    "وضع_الطرف": FunctionSymbol("وضع_الطرف", VOID_TYPE, 2), # pinMode
    "اكتب_رقمي": FunctionSymbol("اكتب_رقمي", VOID_TYPE, 2), # digitalWrite
    "اقرا_رقمي": FunctionSymbol("اقرا_رقمي", INT_TYPE, 1), # digitalRead
    "اكتب_تناظري": FunctionSymbol("اكتب_تناظري", VOID_TYPE, 2), # analogWrite
    "اقرا_تناظري": FunctionSymbol("اقرا_تناظري", INT_TYPE, 1), # analogRead
    "انتظر": FunctionSymbol("انتظر", VOID_TYPE, 1), # delay
    "الزمن_الحالي": FunctionSymbol("الزمن_الحالي", INT_TYPE, 0), # millis
}
```

دوال مكتبة السيريال — تمت إعادة هيكلتها في المرحلة الثالثة إلى ثلاث دوال منفصلة حسب النوع لضمان أمان الأنواع على معمارية AVR وتجنب مشاكل الـ ABI:

```
LIBRARIES = {
    "سيريال": {
        "سيريال_ابدا": FunctionSymbol("سيريال_ابدا", VOID_TYPE, 1), # Serial.begin
        "سيريال_اطبع_صحيح": FunctionSymbol("سيريال_اطبع_صحيح", VOID_TYPE, 1), # c_serialPrintInt
        "سيريال_اطبع_عشري": FunctionSymbol("سيريال_اطبع_عشري", VOID_TYPE, 1), # c_serialPrintFloat
        "سيريال_اطبع_نص": FunctionSymbol("سيريال_اطبع_نص", VOID_TYPE, 1), # c_serialPrintString
    },
}
```

4.1.2. التحكم المباشر بمنافذ معالج (ATmega328P Hardware Register Mapping)

```
REGISTERS = {
    "0": "اتجاه_ب", # DDRB
    "0": "منفذ_ب", # PORTB
    "0": "دخل_ب", # PINB
    # يدعم المترجم مجموعات المنافذ وود بنفس الأسلوب
}
```

4.2. جدول الرموز وإدارة النطاقات المعجمية (Symbol Table & Scope Management)

4.2.1. فئات سجلات الرموز

تم تصميم فئات الرموز داخل ملف `semantic/symbols.py` بشكل كائني منظم:

- **Symbol (الفئة الأم):** تخزن الاسم ونوع البيانات.
- **VariableSymbol:** يمثل المتغيرات العادية المفتوحة للكتابة والقراءة.
- **FunctionSymbol:** يمثل الدوال ويخزن نوع الإرجاع وعدد المعاملات (Arity) كمرجع توثيقي. يتحقق المحلل الدلالي الحالي من أن اسم الدالة معرّف في النطاق، فيما يُعدّ التحقق الكامل من الـ Arity ضمن التطويرات المستقبلية.
- **ConstantSymbol:** يمثل الثوابت العادية القياسية وهو للقراءة فقط.

4.2.2. إدارة النطاقات المتداخلة والتظليل

يعتمد النظام على هيكلية القوائم المرتبطة؛ حيث يمتلك كل نطاق قاموساً خاصاً ومؤشراً `enclosing` يشير للنطاق الأب:

- `define(name, symbol)`: تسجل المتغير في النطاق المحلي الحالي فقط.
- `resolve(name)`: تبحث تصاعدياً من النطاق المحلي حتى النطاق العام.
- **التظليل (Shadowing)**: يتيح إعادة تعريف متغير بنفس اسم متغير في النطاق العام دون إطلاق أخطاء.

4.3. نظام الأنواع والتحقق الدلالي

4.3.1. التحقق من توافق الأنواع والترقية التلقائية

- **العمليات الرياضية والترقية التلقائية**: يدعم الترقية التلقائية من صحيح إلى عشري (`INT_TYPE` → `FLOAT_TYPE`) عند جمع نوعين مختلفين.
- **العمليات البتية وعوامل الإزاحة**: يفرض وجوب كون المعاملين من نوع صحيح (`INT_TYPE`) حصراً.
- **العمليات المنطقية**: تخضع لفحص صارم يفرض كون المعاملين من نوع منطقي (`BOOL_TYPE`).

4.3.2. فحص شروط التحكم وجمل الإرجاع

- **التحقق من جمل القفز (اقطع و تجاوز)**: يمتلك المحلل عداداً `loop_depth` يتحقق من صحة استخدام هذه الجمل داخل حلقات التكرار.
- **التحقق الصارم من جمل الإرجاع (ارجع)**: يتحقق من استخدامها داخل الدوال وتطابق النوع المرجع مع نوع الدالة المعلن.

4.3.3. تقنية تسميم الرموز وقمع الأخطاء المتتالية (Symbol Poisoning)

لتجنب فائض رسائل الخطأ الناتجة عن سبب واحد، يطبق المترجم تقنية "تسميم العقدة" (Symbol Poisoning):

```
# تقنية التسميم والقمع القصير للأخطاء اللاحقة
if left_type == ERROR_TYPE or right_type == ERROR_TYPE:
    return ERROR_TYPE
```

بمجرد أن يرى المترجم النوع المسموم `ERROR_TYPE`، يتجاهل فحص العملية ويمرر النوع المسموم صامتاً دون طباعة أخطاء ثانوية مكررة.

5. الواجهة الوسطى وتحسين الكود الوسيط

5.1. محرك توليد الكود الوسيط (LLVM IR Generator)

5.1.1. تهيئة خصائص العتاد المستهدف ومخطط البيانات للـ AVR

```
target datalayout = "e-P1-p:16:8-i8:8-i16:8-i32:8-i64:8-f32:8-f64:8-n8-a:8"
target triple = "avr-atmel-none"
```

- e : ترميز Little-Endian.
- P1 : فضاء عنوان البرنامج (Flash Memory).
- p:16:8 : مؤشرات 16-بت بمحاذاة 8-بت.
- i8:8 حتى i64:8 : محاذاة بحدود بايت واحد لجميع الأنواع الصحيحة.

5.1.2. تمثيل وإدارة الذاكرة للمتغيرات

- المتغيرات العامة:** تُحجز في SRAM خارج نطاق الدوال وتحمل العلامة @ .
- المتغيرات المحلية:** تُخصص داخل مكدس الدالة باستخدام تعليمة `alloca` ويتم التحكم بها عبر `store` و `load` .

5.1.3. ترجمة التعبيرات واستدعاء دوال العتاد المدمجة

```
declare void @c_pinMode(i32, i32) @__nspace(0)
declare void @c_digitalWrite(i32, i32) @__nspace(0)
declare void @c_delay(i32) @__nspace(0)
```

عند العثور على عقدة استدعاء دالة (مثل `انتظر(٣٠)`)، يولد المحرك:

```
tail call @__nspace(0) void @c_delay(i32 30)
```

5.1.4. إدارة مسارات التحكم وحمايات القسمة على صفر

نظراً لأن معالجات الأردوينو 8-بت تقع في انهيار عتادي عند القسمة على صفر، يولد مولد الكود حماية ذكية:

- توليد مقارنة: `icmp eq i32 %divisor, 0`
- في حال الصفر: القفز لكتلة دُعر تستدعي `@panic_div_zero()`
- في حال الأمان: تنفيذ `%result = sdiv i32 %num, %divisor`

5.2. محرك تحسين الكود لـ AVR (The Middle-End Optimizer)

5.2.1. تمريرات التحسين المستندة إلى الحجم والسرعة (-Os Optimization)

```
# ضبط خيارات التحسين لتقليص المساحة البرمجية عتادياً
pto = llvm.create_pipeline_tuning_options(
    speed_level=2, # تحسين السرعة بمستوى متزن
    size_level=1, # التركيز الأقصى على تقليص حجم الملف الثنائي
)
```

تشمل مرشحات التحسين:

- **قمع المتجهات وال SIMD**: إيقاف تمريرات تحسين المتجهات لعدم وجود مسجلات لها في AVR.
- **قمع فك الحلقات المتكررة (Loop Unrolling)**: لتقليص المساحة وحجم القفزات.
- **دمج الثوابت الرياضية (Constant Folding)**: حساب العمليات ذات الأرقام الثابتة أثناء التصريف.

5.2.2. معالجة وحل ثغرة مجمع AVR الحسابية في LLVM (AVR Unsigned Comparison Fix)

تواجه الواجهة الخلفية لـ LLVM AVR خطأ قديماً يسبب انهيار المترجم (Assertion isUIntN(BitWidth, val)) عند توليد مقارنات ثنائية غير إشارية (icmp ult) مع ثوابت كبيرة.

يطبق محرك التحسين خوارزمية `_avr_fix_unsigned_icmp` التي تحول المعادلة رياضياً:

```
icmp ult iW X, CONST == icmp slt iW (X xor SIGNBIT), (CONST xor SIGNBIT)
```

مثال تطبيقي — قبل المعالجة:

```
%7 = icmp ult i32 %3, 255
```

بعد المعالجة الذكية:

```
%0      = add i32 %3, -255
%7.avrbias = xor i32 %0, -2147483648
%7      = icmp slt i32 %7.avrbias, 2147483394
```

الجزء السادس

الواجهة الخلفية وتوليد كود الآلة والربط (The Back-End & Toolchain)

6. الواجهة الخلفية وتوليد كود الآلة والربط

6.1. تهيئة وتوليد لغة التجميع وكود العتاد

6.1.1. ضبط تهيئة فئات آلة الهدف (backend/avr_target.py)

```

llvm.initialize_all_targets()
llvm.initialize_all_asmpinters()

target = llvm.Target.from_triple("avr-atmel-none")
target_machine = target.create_target_machine(
    cpu="atmega328p",
    features="",
    opt=2,
    reloc="static",
    codemodel="small",
)

```

6.1.2. تصدير كود تجميع AVR وتحويله لملفات كائنية (backend/codegen.py)

مثال حقيقي لكود تجميع AVR المولد تلقائياً:

```

; المولد لتحديث السطوع AVR جزء من كود أسميلي
lds r24, السطوع
lds r18, مقدار_التلاشي
add r24, r18
sts السطوع, r24

```

6.2. نظام الربط وتوليد كود الآلة النهائي (Linking & Hex Generation)

6.2.1. دمج وتجميع كود ملف الكائن مع كود النواة

```
avr-gcc -mmcu=atmega328p -Wl,--gc-sections output.o runtime/arduino_glue.o core.a -o firmware.elf
```

خيار الربط `--gc-sections` يحذف أي دوال استوردت من النواة ولكن لم تُستدع في كود المستخدم، مما يقلص حجم الكود النهائي.

6.2.2. تصدير ملف هيكس عتادي (Hex File)

```
avr-objcopy -O ihex -R .eeprom firmware.elf firmware.hex
```

الملف الناتج `firmware.hex` جاهز للرفع الفوري لمعالج الأردوينو.

6.3. أدوات بناء نواة الأردوينو التلقائية (AVR Toolchain)

- `tools/setup_avr_toolchain.py`: يسحب تلقائياً نسخة معزولة من مجتمعات الأردوينو (`avr-gcc` `avr-gcc` `avr-gcc`).
- `tools/build_core.py`: يُجمع ملفات نواة الأردوينو وينتج مكتبة `core.a` للربط.

7. طبقة بيئة التشغيل ومغلفات لغة C++

7.1. معمارية ملف التشغيل الموحد ومغلفات لغة C (runtime/arduino_glue.cpp)

حل مشكلة تداخل الأسماء (Name Mangling) عبر إحاطة كافة دوال الربط بمغلف "C" : extern

```
// runtime/arduino_glue.cpp
#include <Arduino.h>

extern "C" {
    void c_pinMode(int pin, int mode)      { pinMode(pin, mode); }
    void c_digitalWrite(int pin, int val)  { digitalWrite(pin, val); }
    void c_delay(unsigned long ms)         { delay(ms); }
    void c_serialBegin(long baud)          { Serial.begin(baud); }
    void c_serialPrintInt(int val)          { Serial.print(val); }
    void c_serialPrintFloat(float val)      { Serial.print(val); }
    void c_serialPrintString(const char* s) { Serial.print(s); }
}
```

7.2. دالة البدء الرئيسية ونظام إدارة الذعر (main() & Panic Handlers)

```
int main() {
    init();           // تهيئة المؤقتات والمقاطع
    setup();          // دالة اعداد المترجمة
    for (;;) {
        loop();       // دالة تكرار المترجمة
    }
    return 0;
}
```

معالج الذعر عند القسمة على صفر:

```
extern "C" void panic_div_zero() {
    pinMode(13, OUTPUT);
    while (true) {
        digitalWrite(13, HIGH); delay(100);
        digitalWrite(13, LOW);  delay(100);
    }
}
```

تقوم الدالة بتعليق المعالج في حلقة لانهائية وإضاءة الليد الداخلي (Pin 13) بنمط وميض سريع (100 ميلي ثانية) كإشارة تحذيرية.

بيئة التطوير المتكاملة العربية لـ Arduino (The Arabic IDE)

8. بيئة التطوير المتكاملة العربية لـ Arduino

8.1. تصميم واجهة المستخدم والمحرر العربي ثنائي الاتجاه

- واجهة مستخدم عربية بالكامل: واجهة ثنائية الاتجاه (Right-to-Left) تبدأ القوائم ومساحة التحرير من اليمين.
- محرر نصوص برمجي منسق: يحتوي على شريط تفاعلي لأرقام الأسطر وخط برمجي عربي مخصص يمنع التداخل البصري.

8.2. إدارة الملفات، كاش الجلسات والمسارات المفضلة

- المرونة في التعامل مع الملفات: فتح وحفظ الملفات مع التحقق التلقائي من ترميز UTF-8.
- الحفاظ على جلسات العمل: حفظ مسار آخر ملف وسجل الملفات الحديثة والمنفذ التسلسلي الأخير.

8.3. الربط البرمجي مع محرك البناء والرفع التسلسلي عتادياً

```
avrdude -patmega328p -carduino -PCOM5 -b115200 -D -Uflash:w:firmware.hex:i
```

توفر اللوحة الجانبية أدوات تفاعلية للبحث عن المنافذ التسلسلية النشطة، وعند الضغط على زر التشغيل يمر الكود بدورة تعريف وربط وإنتاج ملف Hex ورفع فوراً للمعالج.

واجهة السائق، الاختبارات والرسم البصري (Driver, Testing & Visualization)

9. واجهة السائق، الاختبارات والرسم البصري

9.1. محرك تجميع وإدارة عمليات التصريف الموحد

- سائق الـ AST والتحليل الدلالي main.py: ينسق قراءة الملف (المُعَرَّف حالياً بشكل ثابت على test_arduino.txt)، وتسويته، وإجراء الفحص المعجمي والنحوي والدلالي، وإنتاج كود الـ LLVM IR الخام وحفظه في ملف debug.11. لتصريف ملف مختلف يجب تعديل مسار الملف مباشرةً داخل main.py.
- سائق العمليات الشامل build.py: يتيح للمطورين إجراء كافة العمليات من الـ Terminal.

الأمر	الوظيفة
python build.py setup	تحميل الأدوات وتجميع النواة core.a لمرة واحدة
python build.py compile	تصريف الكود العربي وإنتاج ملف output.o
python build.py link	ربط الأكواد المجمعة وإنتاج firmware.hex
python build.py flash	الرفع التلقائي عتادياً عبر منفذ USB

9.2. اختبارات التحقق الآلية ودراسة الحالات

النموذج	الغرض	الميزات المُختبِرة
samples/blink.txt	وميض الليد	وضع_الطرف ، اكتب_رقمي ، انتظر
samples/serial_hello.txt	طباعة السيريال	سيريال_ابدا ، سيريال_اطبع_نص
samples/panic_test.txt	اختبار الذعر والقسمة على صفر	معالج panic_div_zero()
test_arduino.txt	روبوت مستقل معقد	المحركات، القراءة التناظرية، الشروط، المتغيرات

9.3. الرسم البصري التفاعلي للشجرة المجردة (AST Visualization)

باستخدام الفئة ASTVisualizerVisitor في ملف ast_visualizer.py وقواعد Graphviz، يستطيع المترجم تصدير الشجرة المجردة تفاعلياً كصورة ملونة تبين المتغيرات والدوال والقرارات البرمجية مع تدوين أرقام الأسطر والعمود لكل عقدة.

الجزء العاشر

الخلاصة والآفاق المستقبلية (Conclusion & Future Work)

10. الخلاصة والآفاق المستقبلية

10.1. الخلاصة الفنية للمشروع

يمثل هذا المشروع بيئة أكاديمية وهندسية متكاملة تبرز بوضوح على إمكانية تطوير نظريات المترجمات العميقة والتحليل الدلالي وتصريف لغة الآلة منخفضة المستوى لمعالجات AVR لخدمة لغة برمجية عربية بالكامل مخصصة للتحكم بالأنظمة المدمجة عتادياً.

لقد تكلل المشروع بالنجاح بدءاً من معالجة وتسوية النصوص العربية وتجاوز تضاربات اليونيكود، والتحليل النحوي المتوافق مع شروط LL(1)، مروراً ببناء الـ AST وتفكيك العمليات برمجياً صامتاً، والتحقق الدلالي الصارم مع تطبيق تقنية التسميم الدلالي

الفعالة، وصولاً إلى بناء مولد كود LLVM IR متكامل مع مكتبة نواة عتادية خفيفة وبناء محسّن ذكي يستهدف تقليص مساحة الكود لمعالجات AVR وإنتاج ملفات الـ Hex ورفعها عتادياً بضغطة زر واحدة.

10.2. الآفاق والتطبيقات المستقبلية

- **دعم هياكل البيانات المعقدة:** المصفوفات (Arrays)، والمؤشرات (Pointers)، والبنى المخصصة (Structs) باللغة العربية.
- **التحقق الكامل من الـ Arity:** إضافة فحص كامل لعدد وأنواع معاملات الدوال عند الاستدعاء.
- **بناء مصحح عتادي تفاعلي (Hardware Debugger):** دمج شاشة تتبع دلالية حية في الـ IDE تسمح بمشاهدة قيم المتغيرات ومسجلات المعالج أثناء التشغيل.
- **دعم عائلات جديدة من المعالجات الدقيقة:** ترقية الواجهة الخلفية لتدعم معالجات 32-بت (ARM Cortex، ESP32) لمجالات إنترنت الأشياء (IoT).

مستودعات المشروع على GitHub

ArduinoArabicCompiler — محرك التصريف الأساسي



يحتوي على كامل الواجهة الأمامية والدلالية والخلفية للمترجم: القواعد النحوية ANTLR4، محلل الأنواع، مولد LLVM IR، محسّن AVR، وملفات بيئة التشغيل.

<https://github.com/abdnash-clean/ArduinoArabicCompiler>

arabic_arduino_ide — بيئة التطوير المتكاملة العربية



يحتوي على الواجهة الرسومية ثنائية الاتجاه (RTL) للمترجم: المحرر النصي، إدارة الجلسات، ومنظومة البناء والرفع التلقائي للأردوينو.

https://github.com/saeed-33/arabic_arduino_ide